8-11-2026

Corrección de visibilidad de NNormal

Módulo: Desarrollo de Aplicaciones Web

Alonso Rogado Pastor
2º DAW - IES JULIÁN MARÍAS

Contenido

Resumen del problema 2
Soluciones 2
Solución 1: Degradado oscuro del fondo 2
Solución 2: Bordes en el texto 3
Solución 3: Sombreado de cada línea 4
Solución 4: Panel de sombreado general 4
Conclusión 5

Resumen del problema

La página web de NNormal cuenta con un problema de visibilidad en el carrusel de fotos del Hero. El texto blanco sobre las fotos de nieve, niebla o cielo no cumplen los estándares de la normativa europea de accesibilidad WCAG AA, los cuales se encargan de medir la relación de luz que refleja el texto con cuánta refleja el fondo.

Como se ha establecido la limitación por parte de los entrevistadores de que las imágenes cambian cada semana y no se quieren quitar las fotos ni usar cajas negras opacas, he tenido que llegar a soluciones sencillas que no modifiquen las fotos, que actúen sobre las fotos y que apunten a solucionar el problema lo mejor posible.

Soluciones

Para solucionar este problema, propongo 4 soluciones solo con HTML5 y CSS3. Todas realizan la misma función, oscurecer lo justo detrás del texto, lo único que cambia es la forma de oscurecer.

Solución 1: Degradado oscuro del fondo

He puesto una capa con un degradado que va de negro a transparente entre la foto y el texto.

```css
.hero-banner::after {
  content: "";
  position: absolute;
  inset: 0;
  z-index: 1;
  background: linear-gradient(to top, rgba(0,0,0,0.65) 80%, transparent);
}
@media (min-width: 768px) {
  .hero-banner::after {
    background: linear-gradient(to right, rgba(0,0,0,0.65) 760px, transparent 1100px);
  }
}
```

1. Grado en que soluciona el problema. Soluciona prácticamente todo el problema. Con negro al 65 % el peor fondo posible, que es una foto blanca, da un ratio de 7:1. Solo falla si el texto se sale de la zona oscura. Lighthouse y WAVE no analizan bien las imágenes de fondo.

2. Implementación y mantenimiento. Es fácil de implementar en un componente reutilizable. Dependiendo de dónde está el texto solo habría que cambiar los valores del degradado.

3. Impacto en el diseño. La foto se mantiene intacta fuera de la zona del texto. Es un recurso bastante discreto, así que no causa apenas impacto.

4. Aplicabilidad.

- Contenido: Con foto blanca sigue cumpliendo. Dependiendo de la cantidad de líneas que tenga el texto podría dejar de cumplir y salirse, pero eso puede arreglarse fácilmente.
- Dispositivos: Cambia de dirección con un 'media query' a 768px.
- Navegadores: Funciona en todos los actuales.

Solución 2: Bordes en el texto

He puesto un borde negro alrededor de cada letra.

```css
.hero-banner__content {
  text-shadow: 0 0 4px #000, 0 0 12px #000;
}
```

1. Grado en que soluciona el problema. Soluciona el problema parcialmente. El borde ayuda, pero no he podido calcular un ratio en base a eso.

2. Implementación y mantenimiento. Es la solución más simple de implementar al tener una sola línea y no necesitar ajustes por banner.

3. Impacto en el diseño. Es la solución que menos impacta en la foto. Si fuera más fuerte el borde podría generar algún problema, pero con valores pequeños se puede leer el texto cómodamente.

4. Aplicabilidad.

- Contenido: Con foto blanca ayuda, pero no garantiza una solución profesional. El número de líneas no importa porque la sombra va con cada letra.
- Dispositivos: Funciona en cualquier posición sin tocar nada.
- Navegadores: Funciona en cualquier dispositivo.

Solución 3: Sombreado de cada línea

He puesto un fondo oscuro en cada etiqueta span, como un fondo semitransparente para cada línea. Entre líneas se sigue viendo la foto bien.

```css
.hero-banner__content span {
  background: rgba(0,0,0,0.75);
  padding: 0.05em 0.5em;
  -webkit-box-decoration-break: clone;
  box-decoration-break: clone;
}
.hero-banner__title { line-height: 1.35; }
```

1. Grado en que soluciona el problema. Soluciona el problema completamente. Con negro al 75 % el ratio es de unos 10:1 aunque la foto sea blanca. Los validadores lo entienden mejor porque el fondo está en el propio texto, aunque con transparencia puede salir revisión manual.

2. Implementación y mantenimiento. Es fácil de implementar, pero necesita una etiqueta span dentro de cada texto.

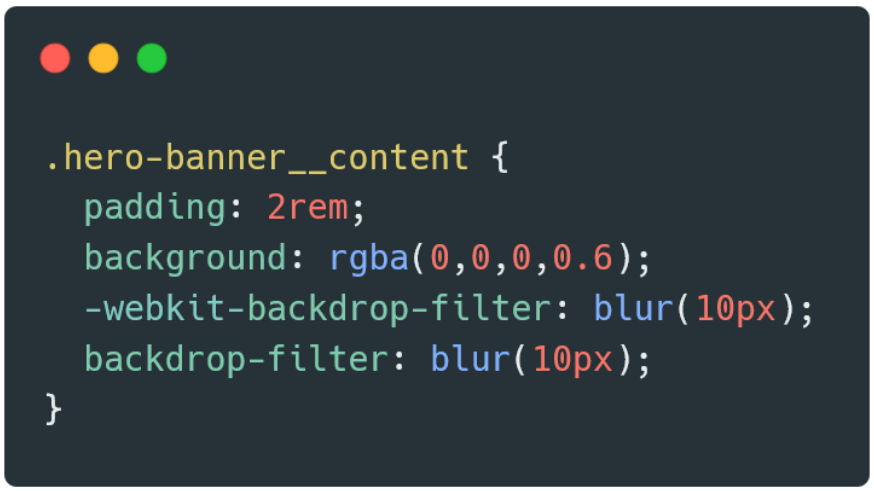
3. Impacto en el diseño. Disrumpe un poco el estilo al introducir fondo negro bajo detrás de cada línea pero esto no afecta a la foto, ya que sigue visible entre líneas.

4. Aplicabilidad.

- Contenido: Con foto blanca cumple. Con distintas líneas, cada línea lleva su propio fondo.
- Dispositivos: Funciona bien, siempre está detrás del texto en cualquier posición.
- Navegadores: Funciona en todos los navegadores pero ‘box-decoration-break’ necesita prefijo ‘-webkit-‘ en algunos. Sin él se ve una caja partida, pero el contraste se mantiene.

Solución 4: Panel de sombreado general

He puesto un fondo negro semitransparente que engloba a todo el párrafo de texto con la foto desenfocada detrás.



```css
.hero-banner__content {
  padding: 2rem;
  background: rgba(0,0,0,0.6);
  -webkit-backdrop-filter: blur(10px);
  backdrop-filter: blur(10px);
}
```

1. Grado en que soluciona el problema. También soluciona el problema completamente. Con negro al 60 %, el peor caso da un ratio de 5.7:1. La garantía viene del color y no del desenfoque, así que funciona aunque el navegador no soporte 'backdrop-filter'. Los validadores se comportan como en la solución 3.

2. Implementación y mantenimiento. Es fácil de implementar ya que es un solo bloque que funciona solo. Solo habría que revisar el padding si cambiara el diseño.

3. Impacto en el diseño. Es la que más cambia visualmente y más se acerca a la solución prohibida de introducir una caja negra opaca detrás del texto. Al ser semitransparente y desenfocada queda mejor, pero puede que la solución no sea válida debido a esas similaritudes.

4. Aplicabilidad.

- Contenido: Con foto blanca cumple. Con 4 líneas el panel crece con el texto.
- Dispositivos: El panel sigue al texto, aunque en móvil ocupa casi todo el ancho.
- Navegadores: Funciona en todos los navegadores ya que 'backdrop-filter' está soportado en todos. Sin él queda el fondo semitransparente.

Conclusión

He hecho una tabla de comparación entre las soluciones:

Solución	Garantiza AA	Facilidad	Impacto visual
Degradado del fondo	Casi siempre	Alta	Bajo

Bordes del texto	No	Muy alta	Muy bajo
Sombreado de línea	Sí	Media	Alto
Sombreado general	Sí	Alta	Medio-alto

Yo elegiría la solución 1 como base, porque es la que menos molesta a la foto y casi siempre cumple. Como reemplazo se podría usar la solución 4 como refuerzo en móvil, donde el texto ocupa más espacio.